Project 1 – Report – INF264

Introduction

This project shows an implementation of a decision tree classifier, which is evaluated on the dataset *churn-data.csv* with 7032 rows, 19 features and a target label of churn or no churn. The first part of the task is building the decision tree classifier with the parameter's *impurity_criterion* and *max_depth* and functions *fit()* and *predict()* using the ID3-algorithm. The second is for model learning and evaluation. The final is for checking feature importance. This report will go over all parts of the projects, explaining the implementation, why certain choices were made and how they were made.

Declaration of use of AI

The LLM ChatGPT from OpenAI was used for quality assurance and troubleshooting for this task (OpenAI, 2026), but was not used for direct code or text generation. All text and code are self-written, and code is understood in accordance with itself and other parts of the program.

Decision Tree

Initialization

The implementation of the decision tree follows the template outlined in the task description quite closely. The decision tree is initialized as a class *DecisionTree* in the file *decision_tree.py.* The constructor is defined with *criterion* and *max_depth* as parameters and giving these the default values of *entropy* and *None* respectively.

Implementation of the *fit()* - function

The implementation of the *fit(X, y)*-function is done by following the ID3-algorithm. To avoid having one massive function, a variety of supporting functions which all go back into building the tree with the given data is utilized.

The first is a recursive function, *build_tree(),* which serves as the primary builder by building one branch at a time and returning once reaching a leaf-node, i.e. when the tree is finished. Determining when a leaf node is reached is checked multiple times:

1. Firstly, when all datapoints have the same label, returning a leaf node with that label.
2. Secondly, if all datapoints have identical feature values, returning a leaf node with the most common label.
3. Thirdly, when recursion reaches max depth, returning a leaf node with the most common label.
4. Lastly, when no best feature index is found or a split gives no additional information, returning a leaf node with the most common label.

If none of these conditions are met, the tree is split at the best feature index, and the function runs recursively on the two halves until leaf nodes are reached on all branches. Finding the most common leaf node is done using the *find_majority()*-function, which checks the value count of all nodes and returns the one with the highest value.

The best feature index is found with the supporting function, *find_max_info_gain()*-function. This function finds the feature with the maximum information gain via the given impurity-measure, being either entropy or Gini index. It looks through all the indices for the features and calculates the information gain for each feature, using find_info_gain(), then returns the maximum information gain value along with the best feature index and the threshold to determine where to divide the tree in *build_tree().* To find the thresholds for the continuous features, the unique values for each feature are sorted and placed halfway between each pair of consecutive values, using the formula $t = \frac{x_i + x_{i+1}}{2}$.

The *find_info_gain()*-function itself works by splitting the features into a left and right side, finding the target labels for the sides and calculating the weighted impurity via the *calculate_impurity()*-function. The impurity of the parent node is also calculated

and it minus the weighted impurity is then returned, unless the lengths of the sides are 0, in which the info gain is also 0 and that is returned as the value. The *calculate_impurity()*-function performs the calculations of the given impurity measure using probabilities as the counts of the unique labels divided by the length of the target labels.

When using entropy, the calculation is done using the formula: $-\sum_{x\epsilon C} p(x) log_2 P(x)$, where p(x) is the probability of label x and C is the total number of labels.

When using the Gini index, the calculation is done using the formula: $1-\sum_{x\epsilon C} p(x)^2$, where p(x) is the probability of label x and C is the total number of labels.

A value closer to 0 is desired for both, meaning all observation belongs to the same class. Values closer to 1 may indicate more mixed labels.

Implementation of the predict() – function

The *predict()*-function is significantly shorter in length of code than fitting the tree, as it only needs to traverse the tree to predict the target label based on input features. For every input observation, the supporting recursive function *predict_node()* is used to traverse the tree, finding the feature index and threshold of the node and running recursively until a leaf node is found and returned. This leaf node is the prediction for the feature. All leaf nodes are added to an array, which is then returned as the prediction in *predict().*

Model Learning and evaluation

The model learning process takes place in *run_experiments.ipynb,* beginning with importing and loading all necessary packages and data. The data is from *churn-data.csv*, which has its categorical features transformed into integer values. The features and target labels are defined as variables X and y, which will be used for training-, validation- and test-data. Printing the feature column names, target label names and their shapes gives this output:

```
Feature columns names: ['gender', 'SeniorCitizen',
Target label name: Churn
X shape: (7032, 19)
y shape: (7032,)
```

Figure 1 - Feature column and target label names with their shapes (feature column names go beyond the image)

This shows us all the feature column names, i.e. the groups that can churn. We also see there are 7032 rows in the dataset.

An important job before starting the model learning process fully was researching the data carefully. The reason this was extra important for the given data is because churn is a binary variable of either 0 or 1, meaning a model that guesses 0 all the time may have an accuracy of above 70% if most customers do not churn. This gives good accuracy but tells us nothing in practice. Therefore, it was important for the data to be properly proportioned in terms of the binary values. First, we start by checking class proportions of all target labels, giving this output:

```
Class proportion:
0    0.734215
1    0.265785
Name: proportion, dtype: float64
```

Figure 2 - Class proportions of target label

This shows us about 73% of the customers in the dataset do not churn and about 27% do. Since not churning is much more prevalent, it was beneficial to stratify the data when splitting into the training-, validation- and test datasets. This strategy for splitting data keeps the class proportions of the original dataset when shuffling. Using the *train-test-split()*-function from sklearn with this strategy, 15% of the data going towards the test-dataset and with a set seed for reproducibility, printing the sample sizes and proportions gives us this output:

```
Training and validation set samples:
5977

Testing set samples:
1055

Training and validation set proportions:
0    0.734148
1    0.265852
Name: proportion, dtype: float64

Testing proportion:
0    0.734597
1    0.265403
Name: proportion, dtype: float64
```

Figure 3 - Sample size and proportions of training- and validation-dataset and test-dataset

The output shows that class proportions have been kept similar for the datasets, ensuring that this does not cause bias later down the line.

The next part is defining two functions to save on duplicate code, being a function for creating and fitting decision trees, called *train_decision_tree()*, and a function for calculating both accuracy-score and F1-score, called *calculate_accuracies()*. It is important to mention here that F1-score will be the primary way to determine goodness of the models, as the data is imbalanced. Accuracy score is included as a useful comparison but can run into the same issue of giving a high score to a model that only guesses a single value for all inputs. The reason why I omitted including recall or precision is because these are used in the calculation of the F1-score, and their trade-off is summarized by it.

Hold-out validation

The primary method of model selection used for this task was hold-out validation. Here the training- and validation-dataset is split into two using the *train-test-split()*-function with a train size that gives a total of 70% training and 15% validation from the total data, with the 15% test remaining untouched. The same set seed and stratifying the data to preserve class proportions is used. Printing the number of samples in each split gives this output:

```
Number of samples in each split:
Training:       4922
Validation:     1055
Testing:        1055
```

Figure 4 - Number of samples in each of the training-, validation- and test-datasets for hold-out validation

This shows the split has correctly divided the data into 70% training, 15% validation and 15% test. To train models and get the accuracy results from them, a function, *get_hold_out_results()*, that uses a nested for-loop to go through combinations of the available hyperparameters, is used. For each criterion and depth, a model is trained and the accuracy results are found by using the functions defined at the start. All the results are added to a data frame for easier viewing and returned. This being a function makes it possible to reuse later when testing against a preexisting classifier.

The given hyperparameters that were available to tune were the impurity-criterion for the tree, i.e. which way to calculate the best features in the tree, and the max depths of the tree, i.e. how many splits along a path from the root are allowed. Since there were only two criteria, entropy and Gini index, both were tested. The max depths could be any positive integer, so a smaller selection of values was used. Given that the more paths in the tree, the risk of overfitting increases, values between 1 and 30 were chosen, with the value *None* serving as a comparison later. Printing all these combinations and sorting by F1-score on validation data gives us this output:

criterion	max_depth	train_acc_score	train_f1_score	val_acc_score	val_f1_score
entropy	1.0	0.734051	0.000000	0.734597	0.000000
entropy	2.0	0.740553	0.597795	0.755450	0.604294
entropy	3.0	0.778748	0.444104	0.790521	0.446115
entropy	4.0	0.778952	0.617171	0.792417	0.623064
entropy	5.0	0.799878	0.576344	0.810427	0.581590
entropy	7.0	0.818773	0.598560	0.800000	0.542299
entropy	10.0	0.862657	0.724531	0.780095	0.562264
entropy	15.0	0.953271	0.909733	0.759242	0.529630
entropy	20.0	0.990857	0.982699	0.746919	0.513661
entropy	30.0	0.998375	0.996935	0.752607	0.529730
entropy	NaN	0.998375	0.996935	0.752607	0.529730
gini	1.0	0.734051	0.000000	0.734597	0.000000
gini	2.0	0.740553	0.597795	0.755450	0.604294
gini	3.0	0.778748	0.444104	0.790521	0.446115
gini	4.0	0.778952	0.617171	0.792417	0.623064
gini	5.0	0.800488	0.580342	0.809479	0.583851
gini	7.0	0.823243	0.623377	0.790521	0.544330
gini	10.0	0.874238	0.752697	0.773460	0.556586
gini	15.0	0.967493	0.936859	0.740284	0.500000
gini	20.0	0.996140	0.992729	0.734597	0.508772
gini	30.0	0.998375	0.996935	0.731754	0.502636
gini	NaN	0.998375	0.996935	0.731754	0.502636

Figure 5 - Different values of impurity-criterion and max depth for hold-out validation

This output shows that the F1-score favors trees with smaller depths, except for 1, as the tree can only make a single split and it becomes much harder to distinguish the classes. Max depth of 3 is also an outlier, possibly explained by the greedy construction of the tree not being able to generalize well on the specific validation data. We also see that the impurity criteria have little effect on the F1-score. The best model has a max depth of 4 and an F1-score on validation data of 0.623064 for both impurity criteria. Plotting the F1-scores on validation, we get these graphs:

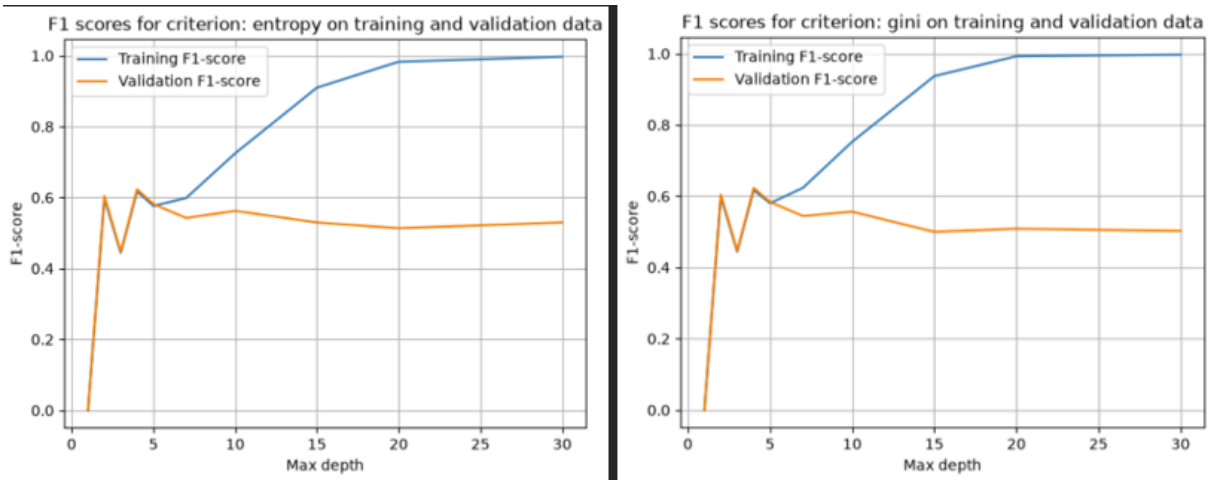


Figure 6 - F1-scores for entropy and Gini index with given max depths on training- and validation-data for hold-out validation

Analyzing the graphs, we can see how F1-score develops with depth. We also see how the score increases on training data as max depth grows, reaching close to 100% accuracy, yet lowers slightly or stays the same on validation data. This gives further evidence of our hypothesis of a larger depth leading into overfitting as the model learns all the noise of the data and is unable to effectively generalize to new data.

We can also plot the accuracy scores:



Figure 7 - Accuracy scores for entropy and Gini index on training- and validation-data for hold-out validation

The plots show the same trend of overfitting as depth increases. The difference between training and validation data may seem extreme here, but that is mostly a product of the y-axis starting at around 0.7.

K-fold cross-validation

To ensure our hyperparameters are well chosen, we can utilize k-fold cross-validation to check if our hyperparameters work well on other combinations of training- and validation-data. This works by segmenting the training- and validation-dataset into N parts, allowing us to train models for the given amount of different combinations of training- and validation-datasets from their common ancestor set. To avoid duplicate code, this was also made into a function, *get_k_fold_results()*.

Similarly to hold-out validation, a stratified method of k-fold with a set seed and shuffle was used. The same nested for-loop was used as well, but a third for-loop is included to find the indices to split at and create the folded training- and validation-

datasets. A decision tree is then fitted to the datasets and mean accuracies, as well as standard deviation, are found from the predictions before being added to a data frame.

The only new choice for hyperparameter tuning here is the amount of folds, as the criteria and depths tested should be the same for both model selection methods. Using 5 folds gives a good balance between runtime and complexity. For 5 folds, this is the output of the accuracy results:

criterion	max_depth	mean_acc_score	std_acc_score	mean_f1_score	std_f1_score
entropy	1.0	0.734148	0.000307	0.000000	0.000000
entropy	2.0	0.743180	0.008871	0.598648	0.017466
entropy	3.0	0.780491	0.008459	0.472017	0.017409
entropy	4.0	0.779821	0.008755	0.485248	0.082235
entropy	5.0	0.785512	0.009831	0.602823	0.025210
entropy	7.0	0.782665	0.008125	0.551518	0.009818
entropy	10.0	0.765435	0.010087	0.543026	0.023546
entropy	15.0	0.738998	0.006425	0.514587	0.011908
entropy	20.0	0.734315	0.008768	0.505908	0.011495
entropy	30.0	0.732140	0.006774	0.503414	0.006671
entropy	NaN	0.732140	0.006774	0.503414	0.006671
gini	1.0	0.734148	0.000307	0.000000	0.000000
gini	2.0	0.742008	0.009186	0.590719	0.027924
gini	3.0	0.780156	0.009823	0.460936	0.037202
gini	4.0	0.781997	0.009755	0.463783	0.036601
gini	5.0	0.778985	0.008171	0.557460	0.078811
gini	7.0	0.781327	0.007755	0.542223	0.022238
gini	10.0	0.758910	0.005159	0.531934	0.018039
gini	15.0	0.733647	0.012371	0.502223	0.031077
gini	20.0	0.730301	0.008897	0.496219	0.015648
gini	30.0	0.727289	0.010874	0.490398	0.016027
gini	NaN	0.727289	0.010874	0.490398	0.016027

Figure 8 - Different values of impurity-criterion and max depth for k-fold cross-validation

Looking at the results, the cross-validation prefers other depths than the hold-out. It also seems that the impurity criterion matters more here. This is easier to spot with plots, so we can visualize the F1-means and standard deviations:

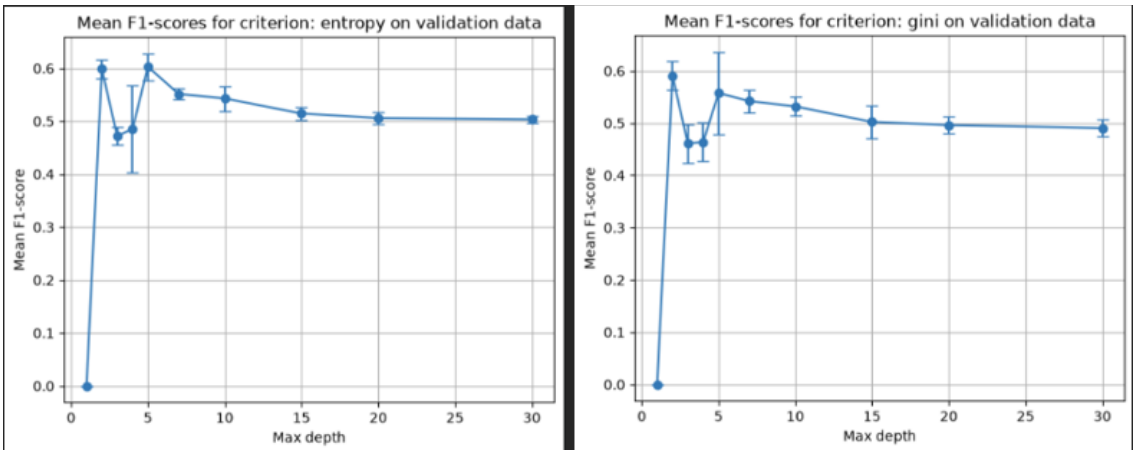


Figure 9 – Mean F1-scores for entropy and Gini index on validation data for k-fold cross-validation

Some similarities with the hold-out plot exist here, such as smaller depths still being preferred for the cross-validation. However, the F1-score is highest for different values. We can also plot the accuracy scores:

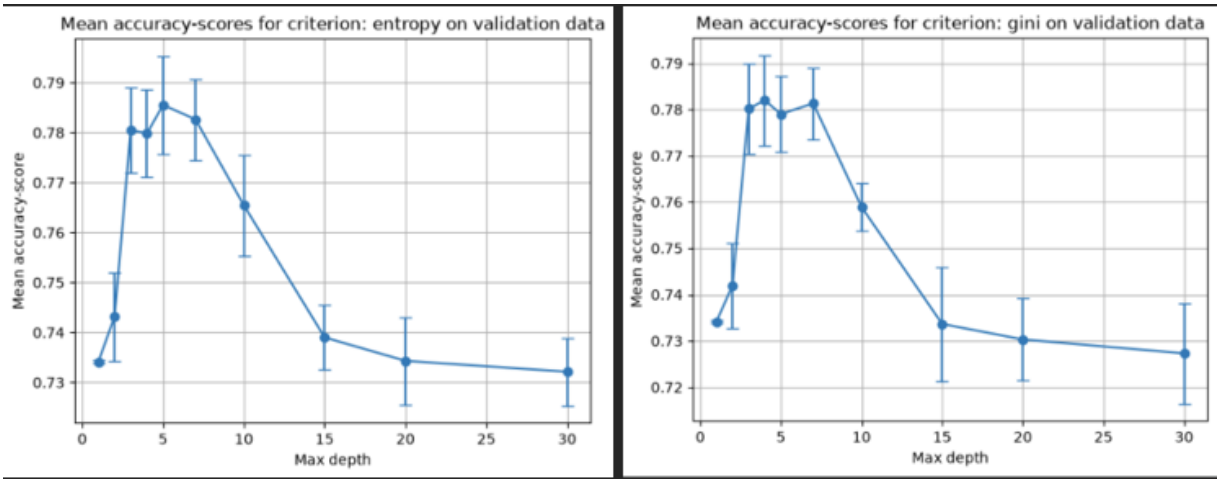


Figure 10 - Mean accuracy-scores for entropy and Gini index on validation data for k-fold cross-validation

These also tell mostly the same story. We can find the best models for each method along with their F1-score, and print them out:

```
Best model using hold-out validation:

criterion           entropy
max_depth               4.0
train_acc_score    0.778952
train_f1_score     0.617171
val_acc_score      0.792417
val_f1_score       0.623064
Name: 3, dtype: object

----------------------------------------
Best model using k-fold cross validation:

criterion           entropy
max_depth               5.0
mean_acc_score     0.785512
std_acc_score      0.009831
mean_f1_score      0.602823
std_f1_score        0.02521
Name: 4, dtype: object
```

Figure 11 - Comparison of the hold-out and k-fold models

Here we see that the best hold-out model uses entropy with a depth of 4, giving an F1-score of 0.623064 on validation data, whereas the best k-fold model uses entropy with a depth of 5, giving a mean F1-score of 0.602823. To see all the differences, we can plot them:

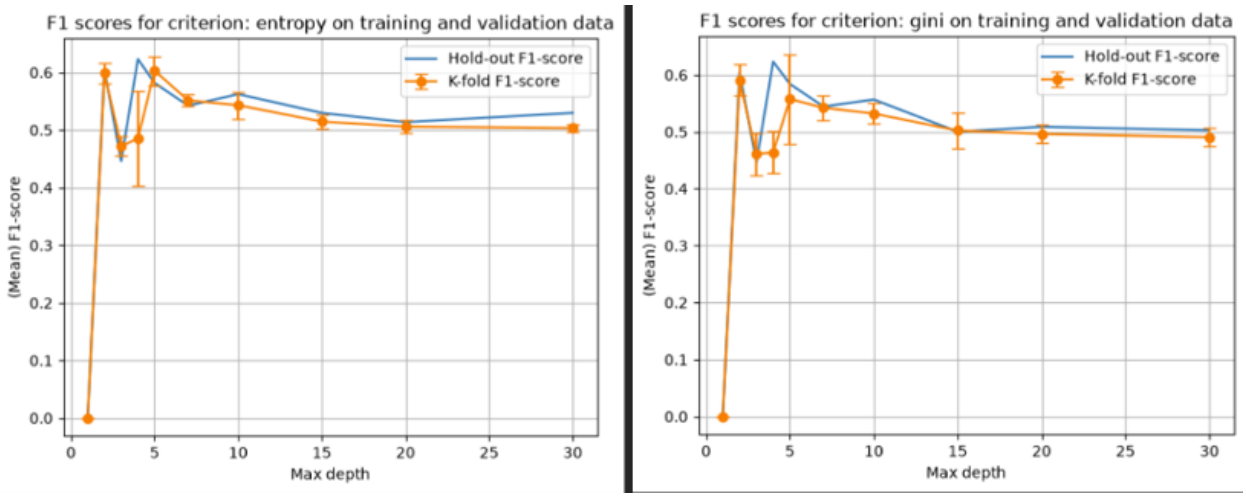


Figure 12 – Comparison of the hold-out and k-fold models on F1-score on validation data

The plots show that both hold-out and k-fold are quite consistent for the same parameters, but it is obvious that they prefer different depths. We cannot make a direct comparison between the two, since they fundamentally differ with k-fold mostly existing as a check for robustness of the hyperparameters. Since the best models differ in their depth, we can also find the k-fold with the same parameters as for the best hold-out model:

```
Best hold_out model:
 criterion           entropy
max_depth               4.0
train_acc_score    0.778952
train_f1_score     0.617171
val_acc_score      0.792417
val_f1_score       0.623064
Name: 3, dtype: object
--------------------------

Accompanying k-fold cross-validation results:
criterion          entropy
max_depth              4.0
mean_acc_score    0.779821
std_acc_score     0.008755
mean_f1_score     0.485248
std_f1_score      0.082235
Name: 3, dtype: object
```

Figure 13 - Accuracies for best hold-out model with accompanying k-fold cross-validation for same parameters

While the accuracy score is quite similar, the drop in F1-score in the k-fold shows that our chosen hyperparameters for the hold-out validation model may be more sensitive to when other data is used to train and validate the model. Since hold-out validation is our primary method of model selection, these are still the hyperparameters we choose to move forward with but keeping in mind that it may not be the absolute best choice for different data sets.

Model evaluation and comparison to preexisting implementations

So far, we have only looked at the models without context of what a good F1-score for the models truly means and if it holds up with a preexisting implementation of a decision tree. The first step is to compare the F1-score of a dummy model using the stratified strategy and set seed with the same training data. Comparing it to the best model from the previous section gives us this output:

```
A dummy classifier with a stratify-strategy has an accuracy-score of 0.5934
 and an F1-score of 0.207 compared to the best tuned model with an
 accuracy-score of 0.7924 and an F1-score of 0.6231
```

Figure 14 - Comparison between dummy classifier and best model

This gives tangible proof that our trained model performs better than a dummy classifier, as the F1-score is substantially higher. We can also note that the accuracy score of the dummy is still high because of the imbalance in the data, reinforcing that the F1-score was the correct measure to use in our model selection.

Before testing our implementation on the test data, we want to compare it to an existing implementation of a decision tree classifier in sklearn. Since we used functions to find accuracy results for both the hold-out and k-fold validations, we can simply slot in the existing classifier as a parameter, as well as using the exact same

hyperparameters. That gives us these outputs:

criterion	max_depth	train_acc_score	train_f1_score	val_acc_score	val_f1_score
entropy	1.0	0.734051	0.000000	0.734597	0.000000
entropy	2.0	0.740553	0.597795	0.755450	0.604294
entropy	3.0	0.778748	0.444104	0.790521	0.446115
entropy	4.0	0.778952	0.617171	0.792417	0.623064
entropy	5.0	0.799878	0.576344	0.810427	0.581590
entropy	7.0	0.818773	0.598560	0.799052	0.541126
entropy	10.0	0.862251	0.724614	0.776303	0.558052
entropy	15.0	0.952255	0.907807	0.765877	0.541744
entropy	20.0	0.990248	0.981524	0.761137	0.536765
entropy	30.0	0.998375	0.996935	0.763981	0.552962
entropy	NaN	0.998375	0.996935	0.763981	0.552962
gini	1.0	0.734051	0.000000	0.734597	0.000000
gini	2.0	0.740553	0.597795	0.755450	0.604294
gini	3.0	0.778748	0.444104	0.790521	0.446115
gini	4.0	0.778952	0.617171	0.792417	0.623064
gini	5.0	0.800488	0.580342	0.809479	0.583851
gini	7.0	0.823243	0.623377	0.790521	0.544330
gini	10.0	0.874238	0.751904	0.776303	0.559701
gini	15.0	0.967290	0.936489	0.746919	0.513661
gini	20.0	0.996546	0.993499	0.743128	0.516934
gini	30.0	0.998375	0.996935	0.741232	0.518519
gini	NaN	0.998375	0.996935	0.741232	0.518519

Figure 15 – Mean accuracy- and F1-scores with standard deviation on different impurity-criteria and max depth for preexisting classifier using hold-out validation

criterion	max_depth	mean_acc_score	std_acc_score	mean_f1_score	std_f1_score
entropy	1.0	0.734148	0.000307	0.000000	0.000000
entropy	2.0	0.743180	0.008871	0.598648	0.017466
entropy	3.0	0.780491	0.008459	0.472017	0.017409
entropy	4.0	0.779821	0.008755	0.485248	0.082235
entropy	5.0	0.785512	0.009831	0.602823	0.025210
entropy	7.0	0.781661	0.009776	0.549244	0.011045
entropy	10.0	0.764933	0.010158	0.543550	0.021640
entropy	15.0	0.743181	0.007525	0.520034	0.016993
entropy	20.0	0.733309	0.009562	0.501842	0.014504
entropy	30.0	0.731302	0.008914	0.500494	0.010949
entropy	NaN	0.730799	0.009512	0.500781	0.012294
gini	1.0	0.734148	0.000307	0.000000	0.000000
gini	2.0	0.742008	0.009186	0.590719	0.027924
gini	3.0	0.780156	0.009823	0.460936	0.037202
gini	4.0	0.781997	0.009755	0.463783	0.036601
gini	5.0	0.778985	0.008171	0.557460	0.078811
gini	7.0	0.779988	0.008445	0.539521	0.021678
gini	10.0	0.757906	0.004789	0.527596	0.014542
gini	15.0	0.731806	0.014979	0.502485	0.026894
gini	20.0	0.730469	0.017573	0.497679	0.025593
gini	30.0	0.727291	0.015595	0.492753	0.021554
gini	NaN	0.726956	0.015726	0.493870	0.017113

Figure 16 - Mean accuracy- and F1-scores on different impurity-criteria and max depth for preexisting classifier using k-fold cross-validation

The outputs seem extremely similar to our custom implementation, so plotting would be redundant. We can first check to see whether the results are the same as the custom implementation when it comes to which parameters hold-out and k-fold prefer:

```
Best preexisting model using hold-out validation:

criterion          entropy
max_depth              4.0
train_acc_score    0.778952
train_f1_score     0.617171
val_acc_score      0.792417
val_f1_score       0.623064
Name: 3, dtype: object

----------------------------------------
Best preexisting model using k-fold cross validation:

criterion          entropy
max_depth              5.0
mean_acc_score    0.785512
std_acc_score     0.009831
mean_f1_score     0.602823
std_f1_score       0.02521
Name: 4, dtype: object
```

Figure 17 - Comparison between hold-out validation and k-fold cross-validation for preexisting classifier

```
Best hold_out model:
 criterion          entropy
max_depth              4.0
train_acc_score    0.778952
train_f1_score     0.617171
val_acc_score      0.792417
val_f1_score       0.623064
Name: 3, dtype: object
--------------------------

Accompanying k-fold cross-validation results:
criterion          entropy
max_depth              4.0
mean_acc_score    0.779821
std_acc_score     0.008755
mean_f1_score     0.485248
std_f1_score      0.082235
Name: 3, dtype: object
```

Figure 18 – Best hold-out model for preexisting classifier on validation data with accompanying k-fold

Seeing that our results are the same as the custom implementation, we can make a direct comparison between the custom and preexisting decision trees:

```
Best model using self-made implementation:

criterion           entropy
max_depth               4.0
train_acc_score    0.778952
train_f1_score     0.617171
val_acc_score      0.792417
val_f1_score       0.623064
Name: 3, dtype: object

----------------------------------------
Best model using preexisting implementation:

criterion           entropy
max_depth               4.0
train_acc_score    0.778952
train_f1_score     0.617171
val_acc_score      0.792417
val_f1_score       0.623064
Name: 3, dtype: object

----------------------------------------
Both implementations have the same F1-score on validation data with F1-score: 0.6231
```

Figure 19 - Comparisons between best models from custom and preexisting decision tree classifiers

The output shows us that both implementations, when trained on the same hyperparameters and data, give identical accuracy. We can interpret this as our model being able to reproduce the performance of the preexisting classifier with the parameters it has, but it is important to mention that the preexisting classifier can be tuned in even more ways, and it ran much faster than the custom implementation. While we can confidently say our custom tree gives a good accuracy- and F1-score, there may be something to desire in terms parameters and runtime.

Doing our final test on the test data for both the custom and preexisting trees gives us this output:

Metric	Self_made_decision_tree	Preexisting_decision_tree
Accuracy_score	0.764929	0.764929
F1_score	0.586667	0.586667

Figure 20 - Test scores for preexisting and custom implementations

This shows us our final model has an F1-score of 0.586667, i.e. we have a moderate balance between precision and recall for the test dataset. The accuracy score of 0.764929 shows that the model predicts churn correctly about 76% of the time, although we must be careful about this number because of the issues mentioned at the start of the section. To check the actual predictions, we can create confusion matrices to give us our true/false positives/negatives on the test data:

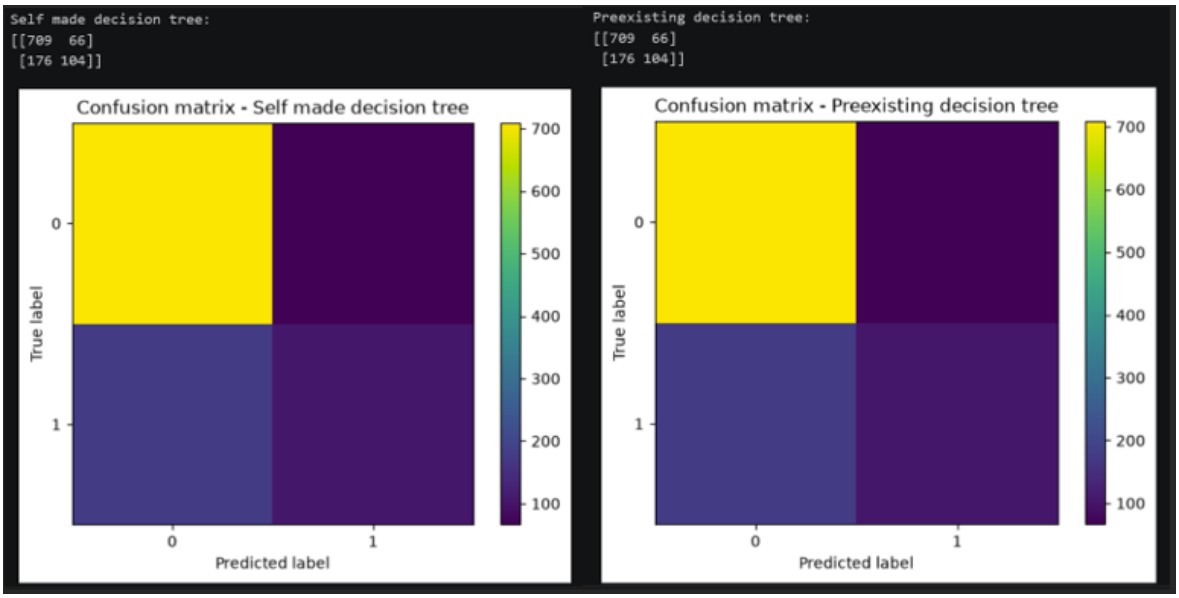


Figure 21 - Confusion matrix for custom and preexisting decision tree on test data

The confusion matrices show that most our predictions are correctly classified as true negatives, i.e. predicting no churn and there being no churn. The other squares include less predictions, i.e. fewer predictions of churn. This points toward the model not being as good at predicting churning and mostly leaning toward predicting no churning all the time, meaning we must take our final accuracy-score with a grain of salt.

Feature Importance

Checking feature importance allows us to see which features have the most impact on model performance, done by using the permutation importance algorithm. We must first implement this algorithm, most easily done by creating a function following the outline from sklearn. This involves computing baseline accuracy for the model, then permuting the data and making new predictions a total of *n_repeats* times. The

importance of each feature is calculated as $i_j = s - \frac{1}{K}\sum_{k=1}^{K} s_{k,j}$, where j is the feature, s is the baseline accuracy of the model, K is the number of repetitions and s_{kj} is the accuracy on the permuted data. With 30 repeats and the same set seed as all other processes, we can then use this function with accuracy-score, and print the output on all the features:

	feature	importance_score
8	OnlineSecurity	4.265403e-02
14	Contract	3.488152e-02
7	InternetService	1.642970e-02
4	tenure	9.889415e-03
2	Partner	3.330669e-16
0	gender	3.330669e-16
1	SeniorCitizen	3.330669e-16
6	MultipleLines	3.330669e-16
5	PhoneService	3.330669e-16
9	OnlineBackup	3.330669e-16
3	Dependents	3.330669e-16
10	DeviceProtection	3.330669e-16
11	TechSupport	3.330669e-16
12	StreamingTV	3.330669e-16
13	StreamingMovies	3.330669e-16
15	PaperlessBilling	3.330669e-16
16	PaymentMethod	3.330669e-16
17	MonthlyCharges	3.330669e-16
18	TotalCharges	3.330669e-16

Figure 22 - Feature importance of all features in test data

To interpret these scores, we need to know what they represent. The decimal number indicates how big the decrease in accuracy is when that feature is permuted, giving us an estimate for how important the feature is for the model performance. Most scores are quite low, but we can see that the four in the top seem to have a bigger hit on performance when permuted. This is easier to spot when visualized with a bar plot:

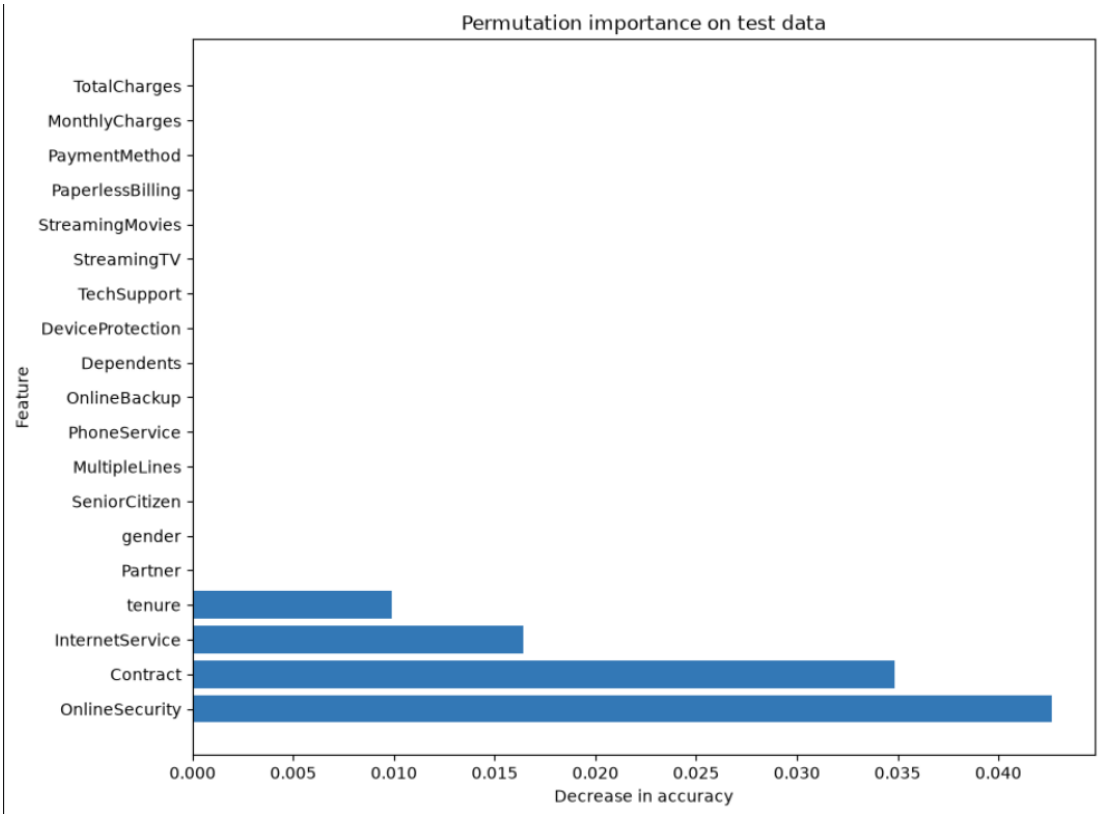


Figure 23 - Decrease in performance for all features when removed from test data

We can conclude that *OnlineSecurity* has the biggest permutation importance for the model on the test data, showing around a 4,3% decrease in accuracy when permuted, followed by *Contract* with about a 3,5% decrease, then *InternetService* with 1.6% and *tenure* with 1%. The rest are negligible.

Some weaknesses of this method of permuting features to see which are the most important for the model is that it is model specific, meaning different models may get different results when permuting. It also is not good at detecting correlated features. Two features could be very important for the model's accuracy, but even when one is permuted, the other may be able to step in as a sort of backup. This would show that the feature was not that important to the model when running the algorithm, when this is not actually the case. Lastly, the process is also computationally expensive. A smaller set such as *churn-data.csv* with only 7032 rows is quickly processed, but a larger dataset could become very expensive because of the required nested for-loop. It does still provide us with a good indication of what features the model relies most on and is a good algorithm to include for this reason.

Conclusion

In conclusion, through this task a custom decision tree implementation has been created using the ID3-algorithm with a given impurity criterion and max depth, before being tested on the dataset *churn-data.csv* and compared to a preexisting implementation of a decision tree classifier from sklearn, then lastly being checked for feature importance. The results of the project show that the implementation was able to receive a moderate F1-score of 0.586667 and accuracy-score of 0.764929 on test data, and also to be near identical to the existing implementation from sklearn, while the features *OnlineSecurity, Contract, InternetService and tenure* were the most impactful features for the model's performance on the test data.